

Package ‘qpmR’

September 29, 2026

Title Quarterly Projection Models for Monetary Policy Analysis

Version 1.1.0

Description An end-to-end implementation of the semi-structural quarterly projection models used in central-bank forecasting and policy analysis systems: model declaration with model-consistent expectations, a generalized Schur solver with Blanchard-Kahn diagnostics following Klein (2000) <[doi:10.1016/S0165-1889\(99\)00045-7](https://doi.org/10.1016/S0165-1889(99)00045-7)>, Kalman filtering and smoothing for latent states such as the output gap and the neutral rate, historical shock decompositions, conditional forecasts that distinguish announced from unanticipated policy paths, an auditable judgment ledger, forecast rounds with revision decompositions, Bayesian estimation with identification diagnostics following Iskrev (2010) <[doi:10.1016/j.jmoneco.2009.12.007](https://doi.org/10.1016/j.jmoneco.2009.12.007)>, and reporting. The canonical small open economy model of Berg, Karam and Laxton (2006) <[doi:10.5089/9781451863413.001](https://doi.org/10.5089/9781451863413.001)> ships as a calibrated template, with extension blocks for disaggregated food inflation and managed exchange rates.

License MIT + file LICENSE

Encoding UTF-8

Language en-GB

LazyData true

RoxygenNote 7.3.3

Depends R (>= 4.1)

Imports Rcpp, QZ, stats, graphics, grDevices, tools, utils

LinkingTo Rcpp, RcppArmadillo

Suggests quarto, testthat (>= 3.0.0), knitr, rmarkdown

Config/testthat/edition 3

VignetteBuilder knitr

URL <https://mustapha-wasseja.github.io/qpmR/>,
<https://github.com/Mustapha-Wasseja/qpmR>

BugReports <https://github.com/Mustapha-Wasseja/qpmR/issues>

NeedsCompilation yes

Author Mustapha Mohammed [aut, cre]

Maintainer Mustapha Mohammed <muswaseja@gmail.com>

Repository CRAN

Date/Publication 2026-09-29 14:10:02 UTC

Contents

add_block	3
add_judgment	4
apply_estimate	5
block_food_cpi	5
block_fx_intervention	7
chart_pack	8
compare_rounds	9
czechia	10
E	11
eigen_table	11
eqs	12
fevd	13
irf	14
judgment_log	15
logLik.qpm_filtration	15
marginal_likelihood	16
model_properties	17
next_quarters	18
nobs.qpm_filtration	18
posterior_forecast	19
priors	20
qpm_block	21
qpm_calibrate	22
qpm_compare_models	23
qpm_condition	24
qpm_counterfactual	25
qpm_decompose	26
qpm_diff	27
qpm_disaggregate	28
qpm_estimate	29
qpm_filter	31
qpm_forecast	32
qpm_identify	33
qpm_lint	34
qpm_model	35
qpm_report	36
qpm_risk	37
qpm_round	38

add_block 3

qpm_rule_eval	39
qpm_scenario	40
qpm_solve	41
qpm_template	42
qpm_use_cpp	43
residuals.qpm_filtration	44
risk_log	44
save_round	45
shocks	46
simulate.qpm_solution	46
state_space	47
steady_state	48
summary.qpm_estimate	49
summary.qpm_filtration	49
var	50
vars	51
vcov.qpm_estimate	51
verify_round	52
write_dynare	53

Index 55

<i>add_block</i>	<i>Apply an extension block to a model</i>
------------------	--

Description

Apply an extension block to a model

Usage

```
add_block(model, block)
```

Arguments

<code>model</code>	A <code>qpm_model</code> .
<code>block</code>	A <code>qpm_block()</code> , or a list of blocks applied in order.

Value

The extended `qpm_model`, with the block recorded in `meta$blocks`.

Examples

```
m <- add_block(qpm_template("bkl"), block_food_cpi(weight = 0.4))
m
```

add_judgment	<i>Add logged judgment to a forecast</i>
--------------	--

Description

Central-bank forecasts are never raw model output: the desk knows about the announced electricity tariff, the tax change, the one-off the model cannot see. `add_judgment()` makes that adjustment a first-class, logged operation: you state the change you want (in percentage points, relative to the current forecast), `qpmR` back-solves the structural shocks needed to support it while keeping the whole forecast model-consistent, records who imposed it and why, and flags judgment that requires implausibly large shocks.

Usage

```
add_judgment(
  fc,
  ...,
  author = "desk",
  rationale = "",
  anticipated = NULL,
  instruments = NULL
)
```

Arguments

<code>fc</code>	A <code>qpm_forecast</code> or a <code>qpm_round</code> .
<code>...</code>	Named adjustments: one argument per variable, each a named vector of <i>additions</i> (percentage points, relative to the current forecast) by period, e.g. <code>pi = c("2027-Q1" = 0.4)</code> .
<code>author</code>	Who is imposing the judgment (logged).
<code>rationale</code>	Why (logged; make it meaningful – the ledger is the audit trail read back before the policy meeting).
<code>anticipated</code>	Expectation mode for the re-solve; defaults to the forecast's current mode, or unanticipated.
<code>instruments</code>	Shocks allowed to move; defaults to the forecast's current instruments, or all shocks.

Details

Judgment entries are stored as absolute targets, so the ledger is replayable; the full set of conditions and judgment is re-solved jointly each time. Inspect the ledger with `judgment_log()`.

Value

The adjusted `qpm_forecast` with the entry appended to its judgment ledger.

Examples

```
sol <- qpm_solve(qpm_template("bk1"))
fc <- qpm_forecast(sol, horizon = 8)
fc2 <- add_judgment(fc, pi = c(h2 = 0.5),
                    author = "desk",
                    rationale = "announced electricity tariff increase")
judgment_log(fc2)
```

apply_estimate	<i>Recalibrate a model at an estimate's point values</i>
----------------	--

Description

Recalibrate a model at an estimate's point values

Usage

```
apply_estimate(est, type = "mean")
```

Arguments

est	A qpm_estimate.
type	Passed to <code>coef.qpm_estimate()</code> .

Value

The estimation model recalibrated at the chosen point estimate (structural parameters and shock sds).

Examples

```
# see ?qpm_estimate
```

block_food_cpi	<i>Disaggregated CPI: food and core inflation</i>
----------------	---

Description

Splits headline inflation into food and core. Food is 30-50 percent of the consumption basket across most of sub-Saharan Africa and South Asia, and a single-inflation model is unusable there: supply shocks to food dominate headline, but monetary policy should look through the relative-price component. Practically every technical-assistance engagement rebuilds this split by hand.

Usage

```
block_food_cpi(
  weight = 0.35,
  persistence = 0.5,
  demand = 0.2,
  passthrough = 0.25,
  correction = 0.1,
  sd = 3
)
```

Arguments

weight	Food share of the CPI basket (w_{food}).
persistence	Food inflation persistence (f_1).
demand	Output-gap coefficient in food inflation (f_2).
passthrough	Real-exchange-rate coefficient in food inflation (f_3); normally larger than core's, food being more tradable.
correction	Error-correction speed on the relative food price (f_4); must be positive for the relative price to be pinned down.
sd	Standard deviation of the food supply shock.

Details

The block replaces headline inflation with an identity and adds:

- `pi_core` — the Phillips curve, now for core inflation (it keeps the template's b_1 , b_2 , b_3 and the `eps_pi` shock);
- `pi_food` — food inflation: its own persistence, expectations of headline, demand, a stronger exchange-rate pass-through than core, and error correction on the relative food price;
- `rp_food` — the relative food price gap, which accumulates the food-core inflation differential and mean-reverts through f_4 .

Headline is $\pi = w_{\text{food}} * \pi_{\text{food}} + (1 - w_{\text{food}}) * \pi_{\text{core}}$, so everything downstream (the 4-quarter average, the Fisher equation, the policy rule) continues to use headline. To target core instead, replace the policy rule with another block.

Value

A `qpm_block()`.

Examples

```
m <- add_block(qpm_template("bkl"), block_food_cpi(weight = 0.45))
sol <- qpm_solve(m)
plot(irf(sol, shock = "eps_pi_food"), vars = c("pi_food", "pi", "pi_core", "i"))
```

 block_fx_intervention *Foreign-exchange intervention (managed float)*

Description

Turns the template's floating exchange rate into a managed one. A leaning-against-the-wind rule responds to the real exchange rate gap, and intervention enters the UIP block directly, so the same model spans a continuum of regimes: $\text{intensity} = 0$ is a free float, moderate values a managed float, and large values approach a peg. Program countries — where reserves, not just the policy rate, are the operative instrument — live in the middle of that range.

Usage

```
block_fx_intervention(intensity = 1, persistence = 0.6, sd = 1)
```

Arguments

intensity	Scales both the intervention response to the RER gap and its effect on the exchange rate. 0 reproduces a free float.
persistence	Persistence of intervention (h_1).
sd	Standard deviation of the discretionary intervention shock.

Details

fx_int is intervention intensity, positive meaning sales of foreign exchange in support of the domestic currency (which appreciates the real exchange rate, lowering q).

Value

A `qpm_block()`.

Examples

```
float <- qpm_solve(qpm_template("bkl"))
managed <- qpm_solve(add_block(qpm_template("bkl"),
                             block_fx_intervention(intensity = 1)))
# the same risk-premium shock moves the exchange rate less under management
```

chart_pack

The standard forecast-round chart pack

Description

Produces the chart set a forecast round is discussed from: the forecast fans, the filtered latent states, the historical shock decomposition of inflation, and the model's monetary transmission. Output is a multi-page PDF by default, or numbered PNGs.

Usage

```
chart_pack(
  round,
  file = NULL,
  charts = NULL,
  vars = NULL,
  width = 9,
  height = 6.5
)
```

Arguments

round	A qpm_round.
file	Output file. A .pdf extension gives one multi-page document; a .png extension gives numbered files (name-1.png, ...). NULL draws to the current device.
charts	Which charts to include, from "forecast", "gaps", "decomposition", "transmission". Default: all four.
vars	Variables for the forecast page. Default: the model's headline set, whichever of π_4/π , i , y_{gap} , q exist.
width, height	Page size in inches.

Value

The output path (invisibly), or NULL when drawing to the current device.

Examples

```
m <- qpm_template("bkl")
obs <- simulate(qpm_solve(m), nsim = 40, seed = 1, burn = 20)
obs$period <- next_quarters("2016-Q1", 40)
r <- qpm_round("demo", m, obs[, c("period", "pi", "i", "q")], horizon = 8)
pdf_path <- file.path(tempdir(), "pack.pdf")
chart_pack(r, pdf_path)
```

compare_rounds

*Compare two forecast rounds: the revision decomposition***Description**

Decomposes the forecast revision between two rounds – "inflation for 2027-Q1 is 0.4pp higher than we said in June: why?" – into the contributions of new data (outturns), data revisions, calibration changes, changed conditions, and judgment, by re-running the full pipeline swapping one ingredient at a time. The contributions telescope, so they sum to the total revision exactly; the final step is verified against the new round's archived forecast.

Usage

```
compare_rounds(old, new, variables = NULL, store = "rounds")
```

```
## S3 method for class 'qpm_revision'
plot(x, variable = NULL, ...)
```

Arguments

old, new	qpm_round objects (or names to <code>load_round()</code> from store).
variables	Variables to decompose (default: all).
store	Round store used when old/new are names.
x	A qpm_revision.
variable	Variable to plot.
...	Unused.

Details

Requirements: both rounds must use the same model structure and the same observables, and their data must carry "YYYY-Qq" period labels so the calendars align. Comparison covers the calendar quarters both rounds forecast.

Value

An object of class qpm_revision: a data frame with one row per variable and overlap period, columns old, new, total, and the five contributions. Print shows waterfall tables; plot() draws stacked contribution bars across the overlap.

Examples

```
m <- qpm_template("bkl")
obs <- simulate(qpm_solve(m), nsim = 44, seed = 1, burn = 20)
obs$period <- next_quarters("2015-Q4", 44)
rA <- qpm_round("June", m, obs[1:40, c("period", "pi", "i", "q")], horizon = 8)
rB <- qpm_round("September", m, obs[, c("period", "pi", "i", "q")], horizon = 8)
```

```

rB <- add_judgment(rB, pi = stats::setNames(0.3, rB$forecast$periods[2]),
                  author = "desk", rationale = "tariff")
rev <- compare_rounds(rA, rB)
rev

```

czechia

Czech quarterly macroeconomic dataset

Description

Quarterly data for Czechia, 1996Q1 onward, in the units and sign conventions of the `qpm_template()` model, ready for `qpm_filter()`. Czechia is the canonical FPAS economy: a small open inflation targeter (since 1998) with three decades of clean data, a large disinflation, a currency-crisis start, the GFC, a floor episode, COVID, and the 2022 inflation wave.

Usage

```
czechia
```

Format

A data frame with one row per quarter and columns:

period Quarter label, "1996-Q1" style.

pi CPI inflation, QoQ annualised percent, seasonally adjusted with `stats::stl()` on the quarterly log index.

pi4 CPI inflation, year on year percent (no adjustment needed).

i 3-month PRIBOR, percent p.a., quarterly average – a proxy for the CNB policy rate.

q Real CZK/EUR exchange rate, 100 times log, CPI-based, normalized so the 2015 average is zero; an increase is a real depreciation of the koruna.

dy_obs Real GDP growth, QoQ annualised percent, from seasonally and calendar adjusted chain-linked volumes.

istar 3-month EURIBOR, percent p.a., quarterly average.

pistar Euro-area HICP inflation, year on year percent.

Details

Missing values are genuine ragged edges (e.g. the euro exists only from 1999, so `q`, `istar` start later); `qpm_filter()` handles them. Observe `pi4` or `pi`, not both – they are linked by an identity and the filter will report the collinearity.

Source

Compiled by `data-raw/czechia.R` from FRED series CLVMNACSCAB1GQCZ (Eurostat national accounts), CZECPALLMINMEI, IR3TIB01CZM156N, IR3TIB01EZM156N (OECD Main Economic Indicators), CP0000EZ19M086NEST (Eurostat HICP), and the ECB reference exchange rate EXR/Q.CZK.EUR.SP00.A. Retrieved 2026-08-22.

Examples

```
head(czechia)
m <- qpm_calibrate(qpm_template("bkl", trends = "rw"),
                  pi_tar = 2, istar_ss = 2, pistar_ss = 2, prem_ss = 1)
cz <- czechia[czechia$period >= "1999",
             c("period", "pi4", "i", "q", "dy_obs", "istar", "pistar")]
fit <- qpm_filter(m, cz)
fit
```

E

Expectations operator (equation syntax only)

Description

E() marks model-consistent expectations inside `eqs()` declarations, e.g. E(pi[+1]). It is parsed by `qpm_model()` and never evaluated as an ordinary function.

Usage

E(x)

Arguments

x A variable reference such as pi[+1].

Value

No return value; calling E() outside of model equations is an error by design.

eigen_table

Generalized eigenvalues of a solved model

Description

Generalized eigenvalues of a solved model

Usage

eigen_table(x)

Arguments

x A qpm_solution.

Value

A data frame with the moduli of the generalized eigenvalues of the model companion pencil, sorted ascending, with stability flags.

Examples

```
head(eigen_table(qpm_solve(qpm_template("bkl"))))
```

eqs

Declare model equations

Description

Each equation is a two-sided formula. Lags and leads use index notation: $x[-1]$ is the first lag, $x[+1]$ the first lead. Wrap expectations in $E()$: $E(\pi[+1])$ is the model-consistent expectation of next quarter's inflation. Longer lags and leads (e.g. $E(\pi_4[+4])$) are handled automatically via auxiliary state variables.

Usage

```
eqs(...)
```

Arguments

```
...           Two-sided formulas.
```

Value

A list of formulas.

Examples

```
eqs(
  pi ~ b1 * pi[-1] + (1 - b1) * E(pi[+1]) + b2 * y_gap + eps_pi
)
```

fevd	<i>Forecast error variance decomposition</i>
------	--

Description

Splits the forecast error variance of each variable at each horizon into the contributions of the structural shocks — "how much of inflation uncertainty two years out is the cost-push shock?". For

$$x_t = Px_{t-1} + Qe_t, e_t \sim N(0, S)$$

the h-step forecast error variance is

$$V_h = \sum_{j < h} P^j Q S Q' (P^j)'$$

and the share of shock k is the same sum with only column k of Q active. Shares sum to one across shocks for every variable and horizon.

Usage

```
fevd(x, ...)
```

```
## S3 method for class 'qpm_solution'
```

```
fevd(x, horizon = 24, vars = NULL, shocks = NULL, ...)
```

```
## S3 method for class 'qpm_fevd'
```

```
plot(x, var = NULL, drop_zero = TRUE, ...)
```

Arguments

x	A qpm_solution, or a qpm_model (solved first).
...	Passed to methods.
horizon	Largest forecast horizon.
vars	Variables to include; default all declared variables.
shocks	Shocks to include; default all.
var	Variable to plot.
drop_zero	Omit shocks that never contribute.

Details

Shares remain well defined when the model has unit roots, even though the variances themselves grow without bound.

Value

A data frame of class qpm_fevd in long format with columns variable, shock, horizon, share and variance. print() shows the dominant shocks; plot() draws stacked shares.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
fv <- fevd(sol, horizon = 20)
fv
plot(fv, var = "pi")
```

irf	<i>Impulse response functions</i>
-----	-----------------------------------

Description

Impulse response functions

Usage

```
irf(x, ...)
```

S3 method for class 'qpm_solution'

```
irf(x, shock = NULL, horizon = 24, size = NULL, vars = NULL, ...)
```

Arguments

x	A qpm_solution from qpm_solve() .
...	Passed to methods.
shock	Shock name(s); default all shocks.
horizon	Number of quarters after impact.
size	Shock size(s). Default is one standard deviation (from the model's sigma); a named vector or a single number can override.
vars	Variables to include; default all declared (non-auxiliary) variables.

Value

A data frame of class qpm_irf in long format with columns shock, variable, horizon, value (deviations from steady state). Printing shows peak effects; plot() draws panel charts.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
ir <- irf(sol, shock = "eps_i", horizon = 16)
ir
plot(ir, vars = c("pi", "y_gap", "i", "q"))
```

judgment_log

Print a forecast's judgment ledger

Description

Print a forecast's judgment ledger

Usage

```
judgment_log(fc)
```

Arguments

fc A qpm_forecast with judgment applied.

Value

The ledger data frame, invisibly.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
fc <- add_judgment(qpm_forecast(sol, horizon = 8), pi = c(h2 = 0.5),
                  author = "desk", rationale = "tariff")
judgment_log(fc)
```

logLik.qpm_filtration *Log-likelihood of a filtration or an estimate*

Description

For a qpm_filtration this is the Kalman-filter log-likelihood of the data under the calibrated model, with zero degrees of freedom (nothing was estimated). For a qpm_estimate it is the log-likelihood at the posterior mode (or at the maximum for method = "mle"), with degrees of freedom equal to the number of estimated parameters — so `stats::AIC()` and `stats::BIC()` work.

Usage

```
## S3 method for class 'qpm_filtration'
logLik(object, ...)

## S3 method for class 'qpm_estimate'
logLik(object, ...)
```

Arguments

object A qpm_filtration or qpm_estimate.
 ... Unused.

Value

An object of class logLik.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
obs <- simulate(sol, nsim = 40, seed = 1, burn = 20)
fit <- qpm_filter(sol, obs[, c("period", "pi", "i", "q")])
logLik(fit)
```

marginal_likelihood	<i>Marginal likelihood of an estimated model</i>
---------------------	--

Description

Computes the log marginal likelihood $\log p(y)$ of a Bayesian estimate – the quantity whose differences across models on the same data are log Bayes factors. Two estimators are reported:

Usage

```
marginal_likelihood(est, taus = seq(0.1, 0.9, by = 0.2))
```

Arguments

est A qpm_estimate with method = "bayes".
 taus Truncation probabilities for the modified harmonic mean.

Details

- **Modified harmonic mean** (Geweke 1999) from the posterior draws, computed for a range of truncation probabilities; a small spread across truncations indicates a reliable estimate.
- **Laplace approximation** at the posterior mode in transformed space (parametrization-invariant because the Jacobian is included), with a numerically differenced Hessian.

Truncated priors created with `truncate()` are renormalized numerically at construction, so they contribute proper densities here.

Value

An object of class qpm_logml: \$logml (harmonic-mean estimate, averaged over taus), \$by_tau, \$laplace, and the spread across truncations.

References

Geweke, J. (1999). Using simulation methods for Bayesian econometric models. *Econometric Reviews*, 18(1), 1-73.

Examples

```
m <- qpm_model(variables = vars(x = "x"), shocks = shocks(e),
               equations = eqs(x ~ rho * x[-1] + e),
               params = list(rho = 0.5))
obs <- simulate(qpm_solve(qpm_calibrate(m, rho = 0.8)), nsim = 120, seed = 1)
est <- qpm_estimate(m, obs, priors(rho = beta(0.5, 0.2)),
                  iter = 300, chains = 2, seed = 2, verbose = FALSE)
marginal_likelihood(est)
```

model_properties	<i>Model-implied moments, and how they compare with the data</i>
------------------	--

Description

The standard calibration check: does the model reproduce the volatilities and persistence actually observed? Reports the population standard deviation and autocorrelations implied by the solved model — from the stationary covariance $V = PVP' + QSQ'$ and $corr_k = diag(P^k V) / diag(V)$ — next to the same statistics computed from data, plus the shock that accounts for most of each variable's unconditional variance.

Usage

```
model_properties(x, data = NULL, vars = NULL, lags = c(1, 4))
```

Arguments

x	A qpm_solution or qpm_model.
data	Optional data frame of observations in levels (columns named for model variables, an optional period column) whose moments are shown alongside. Missing values are dropped per variable.
vars	Variables to report; default all declared variables.
lags	Autocorrelation orders to report.

Details

Population moments exist only for stationary models. When the model has unit roots (random-walk trends) they are undefined, and the function reports that rather than returning nonsense; use [fevd\(\)](#) and [qpm_filter\(\)](#) diagnostics instead.

Value

An object of class qpm_properties: a data frame with the model and (optionally) data moments.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
model_properties(sol, vars = c("y_gap", "pi", "i", "q"))

# against simulated data
obs <- simulate(sol, nsim = 200, seed = 5, burn = 50)
model_properties(sol, data = obs, vars = c("y_gap", "pi", "i"))
```

next_quarters	<i>Generate consecutive quarter labels</i>
---------------	--

Description

Generate consecutive quarter labels

Usage

```
next_quarters(last, H)
```

Arguments

last	The starting quarter, "YYYY-Qq" style; the sequence begins at the following quarter.
H	Number of quarters to generate.

Value

Character vector of H quarter labels.

Examples

```
next_quarters("2026-Q3", 4)
```

nobs.qpm_filtration	<i>Number of observations</i>
---------------------	-------------------------------

Description

Number of observations

Usage

```
## S3 method for class 'qpm_filtration'
nobs(object, ...)

## S3 method for class 'qpm_estimate'
nobs(object, ...)
```

Arguments

object	A qpm_filtration or qpm_estimate.
...	Unused.

Value

Number of time periods used.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
obs <- simulate(sol, nsim = 40, seed = 1, burn = 20)
nobs(qpm_filter(sol, obs[, c("period", "pi", "i")]))
```

posterior_forecast	<i>Forecast with parameter uncertainty (posterior fan)</i>
--------------------	--

Description

Produces a forecast whose fan bands integrate over the posterior: for each sampled parameter draw the model is re-solved, the data re-filtered (so the initial state reflects that draw), and a path sampled from the resulting Gaussian predictive; pointwise band quantiles are then taken across draws. The bands therefore combine future-shock uncertainty with parameter uncertainty — they are wider than the fixed-parameter fans of [qpm_forecast\(\)](#).

Usage

```
posterior_forecast(est, horizon = 12, ndraws = 200, bands = c(0.5, 0.7, 0.9))
```

Arguments

est	A qpm_estimate (method "bayes").
horizon	Forecast horizon in quarters.
ndraws	Number of posterior draws to propagate.
bands	Coverage levels for the fan.

Value

A qpm_forecast-compatible object (posterior-mean path, quantile bands, smoothed history from the posterior-mean model); \$posterior records the number of draws used.

Examples

```

m <- qpm_model(variables = vars(x = "x"), shocks = shocks(e),
               equations = eqs(x ~ rho * x[-1] + e),
               params = list(rho = 0.5))
obs <- simulate(qpm_solve(qpm_calibrate(m, rho = 0.8)), nsim = 100, seed = 1)
est <- qpm_estimate(m, obs, priors(rho = beta(0.5, 0.2)),
                  iter = 300, chains = 1, seed = 2, verbose = FALSE)
fc <- posterior_forecast(est, horizon = 8, ndraws = 15)
plot(fc, vars = "x")

```

priors

*Declare priors for Bayesian estimation***Description**

One named argument per estimated quantity: a structural parameter name or a shock name (meaning that shock's standard deviation). Everything without a prior stays calibrated.

Usage

```
priors(...)
```

Arguments

... Named prior declarations, e.g. `b1 = beta(0.7, 0.1)`, `c2 = truncate(normal(1.5, 0.25), lower = 1`

Details

Available distributions (usable only inside `priors()`, so base R's `beta()` and `gamma()` functions are never masked):

- `normal(mean, sd)`
- `beta(mean, sd)` — on (0, 1), mean/sd parametrization
- `gamma(mean, sd)` — on (0, Inf)
- `invgamma(mean, sd)` — on (0, Inf); the usual choice for shock sds
- `uniform(min, max)`
- `truncate(d, lower, upper)` — restrict any of the above; the normalizing constant is dropped (harmless for modes and MCMC)

Value

An object of class `qpm_priors`.

Examples

```
p <- priors(
  b1 = beta(0.7, 0.1),
  b2 = gamma(0.25, 0.1),
  c2 = truncate(normal(1.5, 0.25), lower = 1),
  eps_pi = invgamma(1, 0.3)
)
p
```

qpm_block

Model extension blocks

Description

A block is a documented, reusable bundle of variables, shocks, parameters and equations that adapts a template to a country. Blocks are how a technical-assistance engagement customizes the canonical model without forking it: `add_block()` applies one, `qpm_diff()` shows exactly what changed, and the result is an ordinary `qpm_model`.

Usage

```
qpm_block(
  name,
  variables = NULL,
  shocks = NULL,
  params = list(),
  sigma = NULL,
  equations = NULL,
  description = ""
)
```

Arguments

name	Block name, used in printing and in the model's history.
variables	New variables, from <code>vars()</code> (optional).
shocks	New shocks, from <code>shocks()</code> (optional).
params	Named list of new parameters (optional).
sigma	Named vector of standard deviations for the new shocks (optional; default 1).
equations	Equations, from <code>eqs()</code> : replacements for existing variables and definitions for new ones.
description	One-line description of what the block does.

Details

Equations whose left-hand side names a variable that already exists **replace** that variable's equation; equations for newly declared variables are appended. Left-hand sides must be bare variable names.

Value

An object of class `qpm_block`.

See Also

`block_food_cpi()`, `block_fx_intervention()`

Examples

```
b <- qpm_block(
  name = "risk premium shifter",
  params = list(prem_extra = 0),
  equations = eqs(prem ~ rho_prem * prem[-1] +
                 (1 - rho_prem) * (prem_ss + prem_extra) + eps_prem)
)
m <- add_block(qpm_template("bkl"), b)
```

qpm_calibrate

Update a model's calibration

Description

Update a model's calibration

Usage

```
qpm_calibrate(model, ..., sigma = NULL)
```

Arguments

<code>model</code>	A <code>qpm_model</code> .
<code>...</code>	Named parameter values to update. Every name must already exist in the model (typo protection).
<code>sigma</code>	Optional named vector of shock standard deviations to update.

Value

The updated `qpm_model`.

Examples

```
m <- qpm_template("bkl")
m <- qpm_calibrate(m, b2 = 0.3, c2 = 2)
```

qpm_compare_models *Compare the behaviour of two or more models*

Description

`qpm_diff()` compares model *structure* — what a country team changed in the code. This compares model *behaviour*: the transmission of a given shock and the moments each specification implies. When a calibration is revised, both questions matter, and the second is the one a policy audience asks.

Usage

```
qpm_compare_models(models, shock = NULL, vars = NULL, horizon = 20)
```

```
## S3 method for class 'qpm_model_comparison'
plot(x, vars = NULL, ...)
```

Arguments

<code>models</code>	A named list of <code>qpm_model</code> or <code>qpm_solution</code> objects.
<code>shock</code>	Shock whose impulse responses are compared. Default: the first shock common to every model.
<code>vars</code>	Variables to compare. Default: those common to all models, capped at the usual headline set.
<code>horizon</code>	Impulse-response horizon.
<code>x</code>	A <code>qpm_model_comparison</code> .
<code>...</code>	Unused.

Value

An object of class `qpm_model_comparison` holding the impulse responses and, for stationary models, the implied moments.

Examples

```
base <- qpm_template("bk1")
flat <- qpm_calibrate(base, b2 = 0.05) # a much flatter Phillips curve
cmp <- qpm_compare_models(list(baseline = base, flat = flat),
                           shock = "eps_y")
cmp
plot(cmp, vars = c("pi", "i"))
```

qpm_condition

*Conditional forecasts: impose paths, back out the shocks***Description**

Imposes hard conditions on future values of model variables and finds the minimum-norm structural shocks (in standard-deviation units, optionally restricted to instruments) that deliver them. This is how a policy question becomes a forecast: "what if the policy rate is held at 3.5 for four quarters?" or "what paths are consistent with inflation back at target by 2027?".

Usage

```
qpm_condition(fc, ..., anticipated = FALSE, instruments = NULL)
```

Arguments

fc	A qpm_forecast from qpm_forecast() , or a qpm_round (its forecast is conditioned and the round returned).
...	Named conditions: one argument per variable, each a named vector of levels by period, e.g. <code>i = c("2026-Q4" = 3.5, "2027-Q1" = 3.5)</code> or <code>pi4 = c(h4 = 2)</code> .
anticipated	Logical; announced-at-start (TRUE) vs period-by-period surprises (FALSE, default).
instruments	Character vector of shocks allowed to move; default all shocks.

Details

The anticipated switch is the economics: TRUE means the whole conditioned path is announced at the start of the forecast, so expectations react immediately (forward-looking variables move before the conditioning bites); FALSE means the implied shocks arrive as period-by-period surprises. The two produce materially different paths in any forward-looking model, and practitioners frequently do not know which one their tools assume.

Fan bands are recomputed as the Gaussian conditional distribution of the path given the conditions (using all shocks), so conditioned points have (near) zero width. The mean path uses only the chosen instruments; when instruments is restricted, mean and bands answer slightly different questions – see Antolin-Diaz, Petrella and Rubio-Ramirez (2021) for the full treatment, which is on the qpmR roadmap.

Value

The conditioned qpm_forecast, with `$shocks_implied` (raw units), `$shocks_implied_std` (standard deviations), and the conditions recorded. Printing summarizes the implied shocks and flags any larger than two standard deviations.

Examples

```
sol <- qpm_solve(qpm_template("bk1"))
fc <- qpm_forecast(sol, horizon = 8)
hold <- qpm_condition(fc, i = c(h1 = 9.5, h2 = 9.5, h3 = 9.5, h4 = 9.5),
                     anticipated = TRUE, instruments = "eps_i")
hold
```

qpm_counterfactual	<i>Historical counterfactuals</i>
--------------------	-----------------------------------

Description

Rewrites history with some shocks switched off or scaled: "what if the central bank had simply followed its rule through 2022?" is the path implied by setting the policy shocks to zero over that window and re-running the model from the same starting point with all other shocks unchanged.

Usage

```
qpm_counterfactual(fit, shocks, periods = NULL, factor = 0, label = NULL)
```

```
## S3 method for class 'qpm_counterfactual'
plot(x, vars = NULL, ...)
```

Arguments

fit	A <code>qpm_filtration</code> .
shocks	Shocks to modify.
periods	Periods over which to modify them (labels as in the data, or integer indices). Default: the whole sample.
factor	Multiplier applied to the selected shocks; 0 (the default) switches them off entirely, 0.5 halves them.
label	Optional name for the scenario.
x	A <code>qpm_counterfactual</code> .
vars	Variables to plot.
...	Unused.

Details

This differs from `qpm_decompose()`, which attributes the history that happened; here the history is replayed under a different assumption. The counterfactual is only as good as the model's invariance to the intervention — a Lucas-critique caveat that applies to every exercise of this kind and is worth stating in any write-up.

Value

An object of class `qpm_counterfactual` holding the actual and counterfactual paths and their difference.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
obs <- simulate(sol, nsim = 60, seed = 4, burn = 20)
obs$period <- next_quarters("2010-Q4", 60)
fit <- qpm_filter(sol, obs[, c("period", "pi", "i", "q")])
cf <- qpm_counterfactual(fit, shocks = "eps_i",
                        label = "no policy surprises")
cf
plot(cf, vars = c("pi", "i", "y_gap"))
```

qpm_decompose

Historical shock decomposition

Description

Splits the smoothed history of every variable into the additive contributions of each structural shock plus the carry-over of the pre-sample initial state: with smoothed shocks e_t ,

$$a_t = \sum_j c_j(t) + c_0(t), c_j(t) = P c_j(t-1) + Q_j e_{j,t}$$

The contributions sum exactly to the smoothed state (deviations from steady state); this identity is verified internally.

Usage

```
qpm_decompose(fit, vars = NULL)

## S3 method for class 'qpm_decomposition'
plot(x, var = NULL, drop_zero = TRUE, periods = NULL, ...)
```

Arguments

<code>fit</code>	A <code>qpm_filtration</code> from <code>qpm_filter()</code> .
<code>vars</code>	Variables to keep (default: all declared variables).
<code>x</code>	A <code>qpm_decomposition</code> .
<code>var</code>	Variable to plot.
<code>drop_zero</code>	Drop components that never contribute.
<code>periods</code>	Optional integer window of period indices to display (e.g. 81:110 for the last 30 quarters).
<code>...</code>	Unused.

Value

A long data frame of class `qpm_decomposition` with columns `period`, `variable`, `component` (shock names plus "initial"), and `value` (contribution, in deviations from steady state). `plot()` draws a stacked-bar decomposition with the smoothed total overlaid.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
obs <- simulate(sol, nsim = 60, seed = 3, burn = 20)
fit <- qpm_filter(sol, obs[, c("period", "pi", "i", "q")])
dec <- qpm_decompose(fit)
plot(dec, var = "pi")
```

qpm_diff

Compare two models structurally

Description

Shows what a customization actually changed: variables, shocks and parameters added or removed, equations changed, and recalibrations. This is how a model review works when country teams adapt a template.

Usage

```
qpm_diff(old, new, tol = 1e-10)
```

Arguments

`old, new` `qpm_model` objects.
`tol` Relative tolerance for calling a parameter changed.

Value

An object of class `qpm_model_diff`, printed as a report.

Examples

```
qpm_diff(qpm_template("bkl"),
         add_block(qpm_template("bkl"), block_food_cpi()))
```

qpm_disaggregate

*Temporal disaggregation of low-frequency data***Description**

Turns annual series into quarterly ones consistent with the annual totals. Many of the economies these models are built for publish national accounts annually, so a quarterly projection model has to start by constructing quarterly GDP — usually from an indicator such as industrial production, imports or credit.

Usage

```
qpm_disaggregate(
  annual,
  indicator = NULL,
  frequency = 4,
  method = c("denton", "chow-lin"),
  conversion = c("sum", "average"),
  rho = NULL
)

## S3 method for class 'qpm_disaggregation'
plot(x, ...)
```

Arguments

annual	Numeric vector of low-frequency values.
indicator	Optional numeric vector of high-frequency indicator values, length frequency * length(annual). Required for "chow-lin".
frequency	Periods per low-frequency observation (4 for annual-to-quarterly).
method	"denton" or "chow-lin".
conversion	"sum" or "average".
rho	AR(1) coefficient for "chow-lin". NULL estimates it by a grid search on the profile GLS likelihood. Note that rho is identified only from the <i>low-frequency</i> residuals, so a short sample cannot pin it down: with ten annual observations the estimate is typically driven to zero even when the quarterly residual is strongly autocorrelated. Around thirty low-frequency observations are needed before the estimate is informative; supply rho directly when the sample is shorter.
x	A qpm_disaggregation.
...	Unused.

Details

Two standard methods:

- "denton" — Denton-Cholette proportional first differences. Minimises the squared change in the ratio of the quarterly series to the indicator (or, without an indicator, in the series itself), subject to matching the annual figures. Purely a smoothing method: no regression, no parameters.
- "chow-lin" — generalised least squares on the indicator with AR(1) quarterly residuals, distributing the annual residual across quarters. Uses the indicator's regression relationship, so it is the better choice when the indicator genuinely tracks the target.

Both enforce the aggregation constraint exactly: "sum" for flows (annual GDP is the sum of quarters), "average" for stocks and index levels.

Value

An object of class `qpm_disaggregation`: the high-frequency series, the method used and the fitted parameters.

References

Denton, F. T. (1971); Chow, G. C. and Lin, A. (1971).

Examples

```
# annual GDP with a quarterly indicator
set.seed(1)
q_true <- cumsum(rnorm(40, 0.5)) + 100
annual <- colSums(matrix(q_true, nrow = 4))
ind <- q_true + rnorm(40, 0, 1)
d <- qpm_disaggregate(annual, ind, method = "chow-lin")
d
plot(d)
```

qpm_estimate

Estimate model parameters (Bayesian or maximum likelihood)

Description

Estimates any subset of structural parameters and shock standard deviations from data, using the Kalman-filter likelihood of the solved model. `method = "bayes"` finds the posterior mode (in transformed, unconstrained space), seeds an adaptive random-walk Metropolis sampler with the inverse Hessian, and returns posterior draws with split R-hat and effective-sample-size diagnostics. `method = "mle"` uses the same machinery with flat priors on the declared supports.

Usage

```
qpm_estimate(
  model,
  data,
  priors,
```

```

observables = NULL,
measurement_error = 0,
kappa = 1e+06,
method = c("bayes", "mle"),
iter = 4000,
burn = floor(iter/2),
chains = 2,
thin = 1,
seed = NULL,
verbose = TRUE
)

## S3 method for class 'qpm_estimate'
coef(object, type = c("mean", "mode", "median"), ...)

```

Arguments

<code>model</code>	A <code>qpm_model</code> .
<code>data</code>	Data frame in levels (as for <code>qpm_filter()</code>).
<code>priors</code>	A <code>priors()</code> declaration. Names are structural parameters or shock names (meaning that shock's sd). Required for <code>method = "bayes"</code> ; for <code>"mle"</code> it supplies supports (and is otherwise ignored).
<code>observables</code> , <code>measurement_error</code> , <code>kappa</code>	Passed to the filter.
<code>method</code>	"bayes" (default) or "mle".
<code>iter</code>	MCMC iterations per chain (including burn-in).
<code>burn</code>	Burn-in iterations (default <code>iter/2</code>).
<code>chains</code>	Number of chains (run sequentially).
<code>thin</code>	Keep every thin-th post-burn draw.
<code>seed</code>	Optional RNG seed. The previous state of the random number generator is restored on exit, so a seeded call does not disturb the caller's random stream.
<code>verbose</code>	Report progress with <code>message()</code> .
<code>object</code>	A <code>qpm_estimate</code> .
<code>type</code>	Point estimate: posterior "mean", "mode", or "median".
<code>...</code>	Unused.

Details

Draws that violate Blanchard-Kahn (indeterminacy or explosiveness) receive zero posterior weight — the usual truncation of the prior to the determinacy region. Everything without a prior stays calibrated at its current value.

Value

An object of class `qpm_estimate`: posterior draws (natural units), the mode, acceptance rate, split R-hat and effective sample sizes, and the originating model/data. Use `coef()` to extract point estimates and `apply_estimate()` to recalibrate the model.

Examples

```
m <- qpm_model(variables = vars(x = "x"), shocks = shocks(e),
               equations = eqs(x ~ rho * x[-1] + e),
               params = list(rho = 0.5))
obs <- simulate(qpm_solve(qpm_calibrate(m, rho = 0.8)), nsim = 100, seed = 1)
est <- qpm_estimate(m, obs, priors(rho = beta(0.5, 0.2), e = invgamma(1, 0.3)),
                  iter = 300, chains = 2, seed = 2, verbose = FALSE)

est
```

qpm_filter

Estimate latent states from data (Kalman filter/smoother)

Description

Runs the Kalman filter and RTS smoother over the solved model, jointly inferring every latent variable — output gap, neutral rate, equilibrium exchange rate, trend processes — and the historical structural shocks from whatever subset of variables you actually observe. Missing values (ragged edges, gappy series) are handled naturally.

Usage

```
qpm_filter(x, data, observables = NULL, measurement_error = 0, kappa = 1e+06)
```

Arguments

x	A <code>qpm_model</code> (solved internally) or <code>qpm_solution</code> .
data	A data frame in levels (model units). Columns whose names match declared variables are used as observables; an optional period column provides labels. NAs are allowed anywhere.
observables	Optional character vector restricting which columns are treated as observed.
measurement_error	Measurement-error standard deviation(s): scalar or named vector over observables. Defaults to 0.
kappa	Diffuse-prior variance scale used when the model has unit-root (random-walk) trends; see <code>state_space()</code> .

Details

The filter is initialized at the model's stationary distribution (Lyapunov covariance), which is exact for the stationary models qpmR currently supports.

Value

An object of class `qpm_filtration`: smoothed states in levels (`$states`), their standard errors (`$se`), smoothed structural shocks (`$shocks`), the log-likelihood (`$loglik`), innovation diagnostics (`$diag`), and the full expanded-state matrix (`$states_dev`). Feed it to `qpm_decompose()` for historical shock decompositions or to `qpm_forecast()` to forecast from the smoothed current state.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
obs <- simulate(sol, nsim = 60, seed = 3, burn = 20)
fit <- qpm_filter(sol, obs[, c("period", "pi", "i", "q")])
fit
plot(fit, vars = c("y_gap", "r_bar"))
```

qpm_forecast

Model forecast with uncertainty bands

Description

Iterates the solved model forward from an initial state and computes analytic forecast uncertainty from the shock variances,

$$V_h = PV_{h-1}P' + QSQ'$$

giving Gaussian fan bands around the mean path. The result can then be conditioned on assumed paths with [qpm_condition\(\)](#), shifted by shock scenarios with [qpm_scenario\(\)](#), or adjusted with logged judgment via [add_judgment\(\)](#).

Usage

```
qpm_forecast(
  object,
  from = NULL,
  horizon = 12,
  bands = c(0.5, 0.7, 0.9),
  sigma = NULL
)
```

Arguments

object	A <code>qpm_solution</code> .
from	Initial state: a <code>qpm_filtration</code> from qpm_filter() (the smoothed end-of-sample state is used and the smoothed history is kept for plotting), a <code>qpm_sim</code> from simulate() , a full named deviation vector over <code>object\$vars_all</code> , or <code>NULL</code> (steady state).
horizon	Forecast horizon in quarters.
bands	Coverage levels for the fan, e.g. <code>c(0.5, 0.7, 0.9)</code> .
sigma	Optional named vector of shock standard deviations.

Value

An object of class `qpm_forecast`: a list with paths (long data frame: variable, h, mean, and `lo_*/hi_*` per band, in levels), forecast-period labels in `$periods`, plus the machinery needed for conditioning.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
histq <- simulate(sol, nsim = 40, seed = 7, burn = 20)
fc <- qpm_forecast(sol, from = histq, horizon = 12)
fc
plot(fc, vars = c("pi", "i", "y_gap", "q"))
```

qpm_identify

Identification diagnostics (Iskrev-style Jacobian analysis)

Description

Checks, before any estimation is run, whether the chosen parameters can be told apart by the data. Two Jacobians are analysed numerically at the current calibration, in the spirit of Iskrev (2010):

Usage

```
qpm_identify(model, params = NULL, observables = NULL, lags = 3, h = 1e-05)
```

Arguments

model	A qpm_model.
params	Parameters to check: a character vector of structural parameter and/or shock names, or a <code>priors()</code> object (its names are used). Default: all structural parameters.
observables	Observed variables the moment analysis conditions on. Default: all declared variables.
lags	Autocovariance lags in the moment vector.
h	Relative step for the central differences.

Details

- **solution level:** derivatives of the solved transition, shock loading, and observable steady state with respect to the parameters. Rank deficiency here means some parameter movements do not change the model's solution at all.
- **moment level** (stationary models only): derivatives of the observables' first and second moments (means, and autocovariances up to lags). Rank deficiency here means some parameter movements are observationally equivalent in population.

The report names parameters with (numerically) no effect, parameter combinations spanning any null space, and near-collinear pairs of Jacobian columns (correlation above 0.995) that are only jointly identified.

Value

An object of class `qpm_identification` with the ranks, singular values, and flagged parameters; printed as a verdict list.

References

Iskrev, N. (2010). Local identification in DSGE models. *Journal of Monetary Economics*, 57(2), 189-202.

Examples

```
qpm_identify(qpm_template("bkl"),
             params = c("b1", "b2", "b3", "c1", "c2"),
             observables = c("pi", "i", "q", "dy_obs"))
```

qpm_lint

Check a model for common specification problems

Description

Runs static checks (unused shocks/parameters, bare leads outside $E()$, calibrations outside plausible ranges when the template documents them) and then attempts to solve the model, reporting the Blanchard-Kahn outcome. Unit algebra on declared units is on the roadmap.

Usage

```
qpm_lint(model)
```

Arguments

model A qpm_model.

Value

An object of class qpm_lint (a data frame of checks with status ok, note, warn, or fail), printed with markers.

Examples

```
qpm_lint(qpm_template("bkl"))
```

qpm_model

*Define a quarterly projection model***Description**

Declares a linear semi-structural model with model-consistent expectations. Equations are parsed immediately, so unknown symbols and malformed lag/lead references are caught at construction time. Coefficients are extracted from the current calibration when the model is solved with `qpm_solve()`.

Usage

```
qpm_model(
  name = "QPM model",
  variables,
  shocks,
  equations,
  params = list(),
  sigma = NULL,
  meta = list()
)
```

Arguments

name	Model name used in printed output.
variables	Variable declarations from <code>vars()</code> .
shocks	Shock declarations from <code>shocks()</code> .
equations	Equations from <code>eqs()</code> ; one per variable.
params	Named list/vector of parameter values.
sigma	Named vector of shock standard deviations. Defaults to 1 for every shock.
meta	Optional list of metadata (e.g. plausible parameter ranges used by <code>qpm_lint()</code>).

Value

An object of class `qpm_model`.

Examples

```
m <- qpm_model(
  name      = "AR(1) toy",
  variables = vars(x = "A persistent process"),
  shocks    = shocks(eps_x),
  equations = eqs(x ~ rho * x[-1] + eps_x),
  params    = list(rho = 0.8)
)
m
```

qpm_report	<i>Write (and optionally render) a monetary policy report</i>
------------	---

Description

Turns a forecast round into the document a policy meeting is run from: an executive summary with the numbers filled in, the forecast table and fan charts, the filtered gaps, the shock decomposition, the judgment ledger, an optional revision decomposition against the previous round, and a reproducibility appendix.

Usage

```
qpm_report(
  round,
  file,
  compare_to = NULL,
  store = "rounds",
  render = TRUE,
  engine = c("auto", "quarto", "rmarkdown"),
  quiet = TRUE
)
```

Arguments

round	A qpm_round.
file	Output path; required, so that nothing is written unless a location is named. The extension chooses the format (.html, .pdf, .docx), or .Rmd to write the source only.
compare_to	Optional previous qpm_round (or its name) to add a revision-decomposition section.
store	Round store used to resolve compare_to by name.
render	Render the document, or only write the source.
engine	"auto" (Quarto if present, else rmarkdown), "quarto", or "rmarkdown".
quiet	Suppress rendering progress output.

Details

The report is always written as a self-contained .Rmd source, on the principle that institutions replace the template's *text*, not its plumbing: edit the file, re-render, keep the analysis. Rendering additionally needs pandoc (via Quarto or RStudio/Positron); where none is available — air-gapped forecasting machines, bare CI runners — qpm_report() reports that and returns the .Rmd unrendered rather than failing.

Value

The path actually produced (the rendered document, or the .Rmd when rendering was not possible), invisibly.

Examples

```
m <- qpm_template("bkl")
obs <- simulate(qpm_solve(m), nsim = 40, seed = 1, burn = 20)
obs$period <- next_quarters("2016-Q1", 40)
r <- qpm_round("demo", m, obs[, c("period", "pi", "i", "q")], horizon = 8)
src <- qpm_report(r, file.path(tempdir(), "mpr.Rmd"), render = FALSE)
```

qpm_risk

Express a balance of risks (skewed fan charts)

Description

Central banks rarely believe their fan charts are symmetric: the published judgement is usually "risks to inflation are tilted to the upside". `qpm_risk()` applies that judgement, replacing the Gaussian bands of a forecast with two-piece normal bands whose *mode* stays on the model's projection while the *mean* shifts by the stated skew. Total uncertainty is preserved: the variance implied by the model at each horizon is held fixed, so a skew redistributes risk rather than adding it.

Usage

```
qpm_risk(fc, ..., author = "MPC", rationale = "")
```

Arguments

<code>fc</code>	A <code>qpm_forecast</code> , or a <code>qpm_round</code> (its forecast is used).
<code>...</code>	Named skews: one argument per variable, each a named vector of <i>mean minus mode</i> by period, e.g. <code>pi4 = c("2027-Q1" = 0.3)</code> . A single unnamed value applies to every horizon.
<code>author, rationale</code>	Recorded with the risk assessment, as for judgement.

Details

The skew is stated in the variable's own units as *mean minus mode* — a value of 0.3 on inflation means the risks are worth 0.3 percentage points to the upside. Skews may be given for any subset of variables and horizons; anything unstated keeps symmetric bands.

Unlike `add_judgment()`, which moves the projection itself and back-solves the shocks that support it, this changes only the shape of the distribution around an unchanged central path.

Value

The forecast with skewed bands. `$risk` records the skews and `$paths` gains a mode column alongside mean.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
fc <- qpm_forecast(sol, horizon = 8)
risky <- qpm_risk(fc, pi = 0.4, author = "MPC",
                 rationale = "energy prices tilted to the upside")

risky
plot(risky, vars = c("pi", "i"))
```

qpm_round

Forecast rounds: one replayable artifact per forecast

Description

A forecast round binds everything that produced a forecast – the model and its calibration, the data vintage, the filtration, the forecast with its conditions and judgment – into one object that can be saved, reloaded, re-run, and compared with later rounds via [compare_rounds\(\)](#). This is the object an institution archives: next quarter, "why did the forecast move?" is answered from the rounds, not from memory.

Usage

```
qpm_round(
  name,
  model,
  data,
  observables = NULL,
  horizon = 12,
  bands = c(0.5, 0.7, 0.9),
  measurement_error = 0,
  kappa = 1e+06
)
```

Arguments

name	Round name, e.g. "2026-Q3 September".
model	A <code>qpm_model</code> .
data	Data frame in levels with a period column; passed to qpm_filter() . Quarter labels ("2026-Q3" style) are required for cross-round comparison.
observables	Columns treated as observed (default: all columns matching declared variables).
horizon	Forecast horizon in quarters.
bands	Fan coverage levels.
measurement_error, kappa	Passed to qpm_filter() .

Details

`qpm_round()` runs the standard pipeline (solve, filter, baseline forecast). Conditions, scenarios, and judgment are then applied to the round directly: `qpm_condition()`, `qpm_scenario()`, and `add_judgment()` all accept a round and update its forecast.

Value

An object of class `qpm_round` with elements `model`, `data`, `solution`, `fit`, and `forecast`.

Examples

```
m <- qpm_template("bkl")
sol <- qpm_solve(m)
obs <- simulate(sol, nsim = 40, seed = 1, burn = 20)
obs$period <- next_quarters("2016-Q1", 40)
r <- qpm_round("test round", m, obs[, c("period", "pi", "i", "q")],
              horizon = 8)
r
```

qpm_rule_eval

Evaluate alternative policy rules

Description

Answers the question a policy committee actually asks — *what if we responded differently?* — by re-solving the model over a grid of rule parameters and scoring each one by the unconditional loss

$$L = \sum_v w_v \text{var}(v) + \sum_v w_v^d \text{var}(v - v_{-1})$$

computed from the model's stationary covariance rather than by simulation, so it is exact. Tracing the resulting variance pairs gives the inflation-output variability frontier (the Taylor curve).

Usage

```
qpm_rule_eval(model, grid, loss = c(pi = 1, y_gap = 0.5), diff_loss = NULL)
```

```
## S3 method for class 'qpm_rule_eval'
plot(x, xvar = NULL, yvar = NULL, ...)
```

Arguments

<code>model</code>	A <code>qpm_model</code> .
<code>grid</code>	A data frame of parameter values, one row per rule and one column per parameter (e.g. from <code>expand.grid()</code>).
<code>loss</code>	Named weights on the variances of levels, e.g. <code>c(pi = 1, y_gap = 0.5)</code> .
<code>diff_loss</code>	Named weights on the variances of first differences, e.g. <code>c(i = 0.5)</code> to penalise instrument volatility.

x	A qpm_rule_eval.
xvar, yvar	Axes of the frontier. Either a variable name, whose level variance is used, or a scored column name directly — so xvar = "vard_i" traces the classic trade-off against instrument volatility. Defaults to the first two entries in loss.
...	Unused.

Details

Rules that violate Blanchard-Kahn are reported as such rather than dropped: a policy response too weak to deliver determinacy is a finding, not a missing row.

Value

A data frame of class qpm_rule_eval: the grid, the variance of each targeted variable, the loss, and the Blanchard-Kahn outcome.

Examples

```
m <- qpm_template("bkl")
grid <- expand.grid(c2 = c(1.2, 1.5, 2, 3), c3 = c(0, 0.5, 1))
ev <- qpm_rule_eval(m, grid, loss = c(pi = 1, y_gap = 0.5),
  diff_loss = c(i = 0.5))

ev
plot(ev)
```

qpm_scenario

Shock-based alternative scenarios

Description

Adds a specified path of structural shocks to a forecast – "what if oil pushes the exchange rate 10 percent weaker next quarter?" – without any inversion. Under anticipated = TRUE the shock path is announced at the start of the forecast and expectations react ahead of it.

Usage

```
qpm_scenario(fc, shocks, anticipated = FALSE, label = NULL)
```

Arguments

fc	A qpm_forecast.
shocks	Named list: one entry per shock, each a named vector of shock sizes (raw equation units) by period, e.g. list(eps_q = c(h1 = 1.5)).
anticipated	Announced at start (TRUE) or surprises (FALSE).
label	Optional scenario label for printing.

Value

The shifted `qpm_forecast` (bands unchanged: a deterministic scenario shifts the mean, not the uncertainty).

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
fc <- qpm_forecast(sol, horizon = 8)
dep <- qpm_scenario(fc, shocks = list(eps_q = c(h1 = 3)),
  label = "10pct depreciation")
```

qpm_solve

*Solve a model under model-consistent expectations***Description**

Reduces the model to first-order form (adding auxiliary states for lags/leads beyond one quarter), computes the steady state, and solves for the unique stable rational-expectations solution

$$x_t = Px_{t-1} + Qe_t$$

via the generalized Schur (QZ) decomposition (Klein 2000), with full Blanchard-Kahn diagnostics.

Usage

```
qpm_solve(model, tol = 1e-07)
```

Arguments

<code>model</code>	A <code>qpm_model</code> .
<code>tol</code>	Numerical tolerance for the solution residual check.

Value

An object of class `qpm_solution` with elements `P`, `Q` (transition and impact matrices over the expanded state vector), `ss` (steady state), and an eigenvalue table (see [eigen_table\(\)](#)).

References

Klein, P. (2000). Using the generalized Schur form to solve a multivariate linear rational expectations model. *Journal of Economic Dynamics and Control*, 24(10), 1405-1423.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
sol
```

qpm_template

Shipped model templates

Description

"bkl" is the canonical semi-structural small-open-economy quarterly projection model in the tradition of Berg, Karam and Laxton (2006, IMF WP/06/80-81): an IS curve, a hybrid Phillips curve, a forward-looking inflation-targeting policy rule, and a dampened (hybrid) UIP block, plus equilibrium-trend and foreign processes, and an observation block for real GDP growth ($dy_{obs} = dy_{bar} + 4 * (y_{gap} - y_{gap}[-1])$), so the model can be filtered on actual national-accounts data without modelling the level of potential output. The default calibration is illustrative, for a higher-inflation emerging economy ("Meridia"); it is not any actual country. See [czechia](#) for a real dataset and a matching recalibration example.

Usage

```
qpm_template(
  name = c("bkl", "bkl_food", "managed_fx"),
  trends = c("stationary", "rw")
)
```

Arguments

name	Template name: "bkl", "bkl_food", or "managed_fx".
trends	"stationary" or "rw"; see Details.

Details

trends selects the equilibrium processes:

- "stationary" (default): AR(1) trends anchored at steady-state parameters; the model is fully stationary.
- "rw": driftless random walks for the equilibrium real exchange rate and potential growth (whose levels are pure normalizations); the neutral rate stays anchored by real interest parity ($r_{star} + prem$), since a free random walk there would leave steady-state gaps indeterminate. The model then has unit roots: free trend levels are normalized to minimum norm in the steady state, and [qpm_filter\(\)](#) switches to diffuse initialization automatically. This is the configuration for real data, where trends drift.

Conventions: gaps in percentage points; inflation QoQ annualised; interest rates in percent per annum; q is 100 times the log real exchange rate, an increase is a real depreciation; dy_{obs} is QoQ annualised real GDP growth.

Country-shaped variants are shipped as shortcuts for the canonical model plus an extension block (see [add_block\(\)](#)):

- "bkl_food" — headline CPI split into food and core ([block_food_cpi\(\)](#)), the configuration for economies where food is a large share of the basket.
- "managed_fx" — a leaning-against-the-wind intervention rule entering the UIP block ([block_fx_intervention\(\)](#)).

Value

A calibrated `qpm_model`.

References

Berg, A., Karam, P., and Laxton, D. (2006). A Practical Model-Based Approach to Monetary Policy Analysis - Overview. IMF Working Paper 06/80; and the companion How-To guide, IMF WP 06/81.

Examples

```
m <- qpm_template("bkl")
summary(m)
m_rw <- qpm_template("bkl", trends = "rw")
```

qpm_use_cpp

Use the compiled Kalman filter

Description

qpmR ships a compiled (C++) Kalman filter and an equivalent reference implementation in R. The compiled one is used by default because estimation runs it once per posterior draw; the R one is kept because the two agreeing to machine precision is what makes the compiled path trustworthy, and it is useful when debugging.

Usage

```
qpm_use_cpp()
```

Details

Set `options(qpmR.use_cpp = FALSE)` to force the R implementation.

Value

TRUE if the compiled filter will be used.

Examples

```
qpm_use_cpp()
```

```
residuals.qpm_filtration
```

One-step-ahead prediction errors and fitted values

Description

`residuals()` returns the filter's one-step-ahead prediction errors for the observed series (`type = "innovation"`), the same divided by their standard deviations (`"standardized"`, which is what the outlier flags use), or the smoothed structural shocks (`"shock"`). `fitted()` returns the one-step-ahead predictions of the observables, so that `observed = fitted + innovation`.

Usage

```
## S3 method for class 'qpm_filtration'
residuals(object, type = c("innovation", "standardized", "shock"), ...)

## S3 method for class 'qpm_filtration'
fitted(object, ...)
```

Arguments

<code>object</code>	A <code>qpm_filtration</code> .
<code>type</code>	Which residuals to return.
<code>...</code>	Unused.

Value

A data frame with a period column and one column per series.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
obs <- simulate(sol, nsim = 40, seed = 1, burn = 20)
fit <- qpm_filter(sol, obs[, c("period", "pi", "i", "q")])
head(residuals(fit, "standardized"))
head(fitted(fit))
```

```
risk_log
```

Print a forecast's balance-of-risks assessment

Description

Print a forecast's balance-of-risks assessment

Usage

```
risk_log(fc)
```

Arguments

fc A qpm_forecast or qpm_round with risks applied.

Value

The risk table, invisibly.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
fc <- qpm_risk(qpm_forecast(sol, horizon = 8), pi = 0.4,
               rationale = "upside energy risk")
risk_log(fc)
```

save_round	<i>Save, load, and list forecast rounds</i>
------------	---

Description

A round store is a plain directory: each round lives in `store/<slug>/` as a self-contained `round.rds` plus human-readable sidecars (`forecast.csv`, `data.csv`, `calibration.csv`, and `judgment.csv` when judgment was applied) so that a round can be audited without R.

Usage

```
save_round(round, store, overwrite = FALSE)
```

```
load_round(name, store = "rounds")
```

```
list_rounds(store = "rounds")
```

Arguments

round	A qpm_round.
store	Directory of the round store. <code>save_round()</code> creates it if missing and has no default, so nothing is written unless you name the location; the reading functions default to a "rounds" directory under the working directory.
overwrite	Allow replacing an existing round of the same name.
name	Round name (or its slug), or a direct path to a round directory or <code>round.rds</code> .

Value

`save_round()` returns the round directory invisibly; `load_round()` returns the `qpm_round`; `list_rounds()` returns a data frame of the store's contents.

Examples

```

m <- qpm_template("bkl")
obs <- simulate(qpm_solve(m), nsim = 40, seed = 1, burn = 20)
obs$period <- next_quarters("2016-Q1", 40)
r <- qpm_round("demo", m, obs[, c("period", "pi", "i", "q")], horizon = 8)
store <- file.path(tempdir(), "rounds")
save_round(r, store)
list_rounds(store)
r2 <- load_round("demo", store)

```

shocks	<i>Declare the structural shocks of a model</i>
--------	---

Description

Accepts bare names or character strings.

Usage

```
shocks(...)
```

Arguments

... Shock names, e.g. shocks(eps_y, eps_pi).

Value

A character vector of shock names.

Examples

```
shocks(eps_y, eps_pi, eps_i)
```

simulate.qpm_solution	<i>Simulate a solved model</i>
-----------------------	--------------------------------

Description

Draws Gaussian shocks with the model's standard deviations and iterates the solved transition. Returns variables in levels (steady state plus simulated deviations). The final full state is stored as an attribute so a simulation can be handed straight to [qpm_forecast\(\)](#).

Usage

```

## S3 method for class 'qpm_solution'
simulate(object, nsim = 40, seed = NULL, sigma = NULL, burn = 0, ...)

```

Arguments

object	A qpm_solution.
nsim	Number of quarters to simulate.
seed	Optional seed for reproducibility. The previous RNG state is restored on exit.
sigma	Optional named vector of shock standard deviations overriding the model's.
burn	Burn-in quarters discarded from the start.
...	Unused.

Value

A data frame of class qpm_sim (period plus one column per declared variable), with attributes state (final expanded state, deviations) and shocks (the drawn shocks).

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
hist <- simulate(sol, nsim = 60, seed = 42, burn = 20)
head(hist)
```

state_space

State-space representation of a solved model

Description

Exposes the exact matrices qpmR itself uses for filtering, so other estimators and filters can be built on top of a solved model. The representation (in deviations from steady state) is

$$a_t = T a_{t-1} + R e_t, e_t \sim N(0, Qc)$$

$$y_t = Z a_t + d + u_t, u_t \sim N(0, H)$$

with d the steady state of the observables and P1 the stationary (Lyapunov) covariance used to initialize the filter.

Usage

```
state_space(solution, observables = NULL, measurement_error = 0, kappa = 1e+06)
```

Arguments

solution	A qpm_solution.
observables	Character vector of observed variables (a subset of the declared variables). Default: all declared variables.
measurement_error	Measurement-error standard deviation(s): a scalar recycled over observables, or a named vector.
kappa	Diffuse-prior variance scale for unit-root directions (only used when the model has unit roots).

Details

For stationary models P1 is the exact stationary covariance. When the model has unit roots (random-walk trends), an approximate diffuse initialization is used: P1 solves the Lyapunov equation for the slightly damped transition $\sqrt{1 - 1/\kappa} * T$, which reproduces the stationary covariance in stable directions and a variance of order κ in unit-root directions, with the exact cross-coupling. Exact Durbin-Koopman diffuse recursions are on the roadmap.

Value

A list with elements T, R, Z, d, Qc, H, P1, vars_all, observables, diffuse, n_unit.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
ss <- state_space(sol, observables = c("pi", "i", "q"))
dim(ss$T); ss$d
```

steady_state

Steady state of a model or solution

Description

Steady state of a model or solution

Usage

```
steady_state(x, ...)
```

Arguments

x	A qpm_model or qpm_solution.
...	Unused.

Value

Named vector of steady-state values for the declared variables.

Examples

```
steady_state(qpm_template("bkl"))
```

summary.qpm_estimate *Summarise an estimate*

Description

Returns the posterior summary as a data frame — prior, mode, mean, standard deviation, credible interval, R-hat and effective sample size — so it can be used programmatically rather than only read.

Usage

```
## S3 method for class 'qpm_estimate'
summary(object, level = 0.9, ...)
```

Arguments

object	A qpm_estimate.
level	Credible level for the interval.
...	Unused.

Value

A data frame, one row per estimated parameter.

Examples

```
m <- qpm_model(variables = vars(x = "x"), shocks = shocks(e),
               equations = eqs(x ~ rho * x[-1] + e),
               params = list(rho = 0.5))
obs <- simulate(qpm_solve(qpm_calibrate(m, rho = 0.8)), nsim = 120, seed = 1)
est <- qpm_estimate(m, obs, priors(rho = beta(0.5, 0.2)),
                  iter = 300, chains = 1, seed = 2, verbose = FALSE)
summary(est)
```

summary.qpm_filtration *Summarise a filtration*

Description

Summarise a filtration

Usage

```
## S3 method for class 'qpm_filtration'
summary(object, ...)
```

Arguments

object	A qpm_filtration.
...	Unused.

Value

A data frame with, for each model variable, whether it was observed and the mean, standard deviation and range of its smoothed path, plus the mean estimation standard error for latent states.

Examples

```
sol <- qpm_solve(qpm_template("bkl"))
obs <- simulate(sol, nsim = 40, seed = 1, burn = 20)
summary(qpm_filter(sol, obs[, c("period", "pi", "i", "q")]))
```

var	<i>Declare a model variable with a label and unit</i>
-----	---

Description

Used inside `vars()` to attach documentation to a variable. Units are stored and displayed; unit algebra (automatic consistency checking inside equations) is on the roadmap.

Usage

```
var(label = "", unit = "", ...)
```

Arguments

label	Human-readable description, e.g. "Output gap" (or numeric data, which is passed to <code>stats::var()</code>).
unit	Unit of measurement, e.g. "pp", "pct pa".
...	Passed to <code>stats::var()</code> in the delegation case.

Details

When given numeric data instead of a character label, `var()` delegates to `stats::var()`, so attaching qpmR does not break variance computations.

Value

An object of class `qpm_var`, or the result of `stats::var()`.

Examples

```
vars(y_gap = var("Output gap", unit = "pp"))
var(rnorm(10)) # still the sample variance
```

vars

Declare the endogenous variables of a model

Description

Declare the endogenous variables of a model

Usage

```
vars(...)
```

Arguments

... Named arguments, one per variable. Each value is a `var()` declaration or a character label. Unnamed character arguments are taken as variable names with empty labels.

Value

A data frame with columns name, label, unit.

Examples

```
vars(
  y_gap = var("Output gap", unit = "pp"),
  pi    = "CPI inflation, QoQ annualised"
)
```

vcov.qpm_estimate

Posterior covariance and credible intervals

Description

Posterior covariance and credible intervals

Usage

```
## S3 method for class 'qpm_estimate'
vcov(object, ...)

## S3 method for class 'qpm_estimate'
confint(object, parm = NULL, level = 0.9, ...)
```

Arguments

object	A qpm_estimate.
...	Unused.
parm	Parameters to report; default all.
level	Credible level.

Value

vcov() returns the posterior covariance matrix of the estimated parameters; confint() returns equal-tailed posterior credible intervals (posterior quantiles, not asymptotic intervals).

Examples

```
m <- qpm_model(variables = vars(x = "x"), shocks = shocks(e),
               equations = eqs(x ~ rho * x[-1] + e),
               params = list(rho = 0.5))
obs <- simulate(qpm_solve(qpm_calibrate(m, rho = 0.8)), nsim = 120, seed = 1)
est <- qpm_estimate(m, obs, priors(rho = beta(0.5, 0.2)),
                  iter = 300, chains = 1, seed = 2, verbose = FALSE)

vcov(est)
confint(est)
```

verify_round

Verify that an archived round still reproduces

Description

Re-runs a saved forecast round from its own contents — model, calibration, data vintage, conditions and judgment — and checks that the published numbers come back. This is the audit an institution needs and that a folder of scripts cannot provide: a round that no longer reproduces is a finding, not a mystery.

Usage

```
verify_round(round, store = "rounds", tol = 1e-06)
```

Arguments

round	A qpm_round, or a round name/path to load from store.
store	Round store used when round is a name.
tol	Absolute tolerance on the forecast paths.

Details

When the round is loaded from a store, the human-readable CSV sidecars are checked against the object too, so a hand-edited audit trail is detected.

Value

An object of class `qpm_verification`: \$ok, the largest deviation, the worst variables, sidecar checks, and the qpmR versions involved.

Examples

```
m <- qpm_template("bkl")
obs <- simulate(qpm_solve(m), nsim = 40, seed = 1, burn = 20)
obs$period <- next_quarters("2016-Q1", 40)
r <- qpm_round("demo", m, obs[, c("period", "pi", "i", "q")], horizon = 8)
verify_round(r)
```

write_dynare

*Export a model to a Dynare .mod file***Description**

Writes the model as Dynare source so that anyone can re-solve it in Dynare and compare. This is deliberate: qpmR's solver is pinned in its own tests to analytic solutions, but "does it agree with Dynare?" is the first question a modeller asks, and the answer should be cheap to check rather than something to take on trust.

Usage

```
write_dynare(model, file = NULL, irf = 40, order = 1, extra = NULL)
```

Arguments

<code>model</code>	A <code>qpm_model</code> .
<code>file</code>	Output path. The default <code>NULL</code> returns the Dynare source as a character vector; a file is written only when a path is given.
<code>irf</code>	Horizon for <code>stoch_simul</code> ; 0 omits impulse responses.
<code>order</code>	Approximation order passed to <code>stoch_simul</code> (the models are linear, so first order is exact).
<code>extra</code>	Character vector of extra Dynare statements appended after <code>stoch_simul</code> , e.g. code to export results.

Details

The original equations are exported, not qpmR's internal first-order system: Dynare creates its own auxiliary variables for long leads and lags, so the two implementations agree only if both handle them correctly. Expectation wrappers are dropped ($E(\pi_t[+1])$ becomes $\pi_t(+1)$), since leads in Dynare are already model-consistent expectations.

Value

The file path invisibly, or the Dynare source as a character vector when `file` is `NULL`.

Examples

```
src <- write_dynare(qpm_template("bk1"), file = NULL)
cat(head(src, 15), sep = "\n")
```

Index

* datasets

- czechia, [10](#)
- add_block, [3](#)
- add_block(), [21](#), [42](#)
- add_judgment, [4](#)
- add_judgment(), [32](#), [37](#), [39](#)
- apply_estimate, [5](#)
- apply_estimate(), [30](#)
- block_food_cpi, [5](#)
- block_food_cpi(), [22](#), [42](#)
- block_fx_intervention, [7](#)
- block_fx_intervention(), [22](#), [42](#)
- chart_pack, [8](#)
- coef(), [30](#)
- coef.qpm_estimate(qpm_estimate), [29](#)
- coef.qpm_estimate(), [5](#)
- compare_rounds, [9](#)
- compare_rounds(), [38](#)
- confint.qpm_estimate
(vcov.qpm_estimate), [51](#)
- czechia, [10](#), [42](#)
- E, [11](#)
- eigen_table, [11](#)
- eigen_table(), [41](#)
- eqs, [12](#)
- eqs(), [11](#), [21](#), [35](#)
- expand.grid(), [39](#)
- fevd, [13](#)
- fevd(), [17](#)
- fitted.qpm_filtration
(residuals.qpm_filtration), [44](#)
- irf, [14](#)
- judgment_log, [15](#)
- judgment_log(), [4](#)

- list_rounds(save_round), [45](#)
- load_round(save_round), [45](#)
- load_round(), [9](#)
- logLik.qpm_estimate
(logLik.qpm_filtration), [15](#)
- logLik.qpm_filtration, [15](#)
- marginal_likelihood, [16](#)
- message(), [30](#)
- model_properties, [17](#)
- next_quarters, [18](#)
- nobs.qpm_estimate
(nobs.qpm_filtration), [18](#)
- nobs.qpm_filtration, [18](#)
- plot.qpm_counterfactual
(qpm_counterfactual), [25](#)
- plot.qpm_decomposition(qpm_decompose),
[26](#)
- plot.qpm_disaggregation
(qpm_disaggregate), [28](#)
- plot.qpm_fevd(fevd), [13](#)
- plot.qpm_model_comparison
(qpm_compare_models), [23](#)
- plot.qpm_revision(compare_rounds), [9](#)
- plot.qpm_rule_eval(qpm_rule_eval), [39](#)
- posterior_forecast, [19](#)
- priors, [20](#)
- priors(), [30](#), [33](#)
- qpm_block, [21](#)
- qpm_block(), [3](#), [6](#), [7](#)
- qpm_calibrate, [22](#)
- qpm_compare_models, [23](#)
- qpm_condition, [24](#)
- qpm_condition(), [32](#), [39](#)
- qpm_counterfactual, [25](#)
- qpm_decompose, [26](#)
- qpm_decompose(), [25](#), [31](#)

qpm_diff, 27
qpm_diff(), 21, 23
qpm_disaggregate, 28
qpm_estimate, 29
qpm_filter, 31
qpm_filter(), 10, 17, 26, 30, 32, 38, 42
qpm_forecast, 32
qpm_forecast(), 19, 24, 31, 46
qpm_identify, 33
qpm_lint, 34
qpm_lint(), 35
qpm_model, 35
qpm_model(), 11
qpm_report, 36
qpm_risk, 37
qpm_round, 38
qpm_rule_eval, 39
qpm_scenario, 40
qpm_scenario(), 32, 39
qpm_solve, 41
qpm_solve(), 14, 35
qpm_template, 42
qpm_template(), 10
qpm_use_cpp, 43

residuals.qpm_filtration, 44
risk_log, 44

save_round, 45
shocks, 46
shocks(), 21, 35
simulate(), 32
simulate.qpm_solution, 46
state_space, 47
state_space(), 31
stats::AIC(), 15
stats::BIC(), 15
stats::stl(), 10
stats::var(), 50
steady_state, 48
summary.qpm_estimate, 49
summary.qpm_filtration, 49

var, 50
var(), 51
vars, 51
vars(), 21, 35, 50
vcov.qpm_estimate, 51
verify_round, 52

write_dynare, 53